Graph Theory Coding Task

Group 6



Group Members :

Lina Fatima Azzahra Badr - 5025251168

Naila Sa'ada Cahyani - 5025251258

A. Introduction

For this coding task, we are tasked with using graph theory to implement graph algorithms in order to solve three tasks:

1. Task 1 (SPPG Simulator)
2. Task 2 (DLH Simulator)
3. Task 3 (Knight's Tour Simulator)

B. System Architecture

We use python for the tasks' programs. We separate the algorithm programs (knight_tour.py and chinese_postman.py) and the data ingestion programs (engine folder). After downloading the repository, the tasks' programs can be run from the repository folder as instructed in the readme file.

C. Task 1

D. Task 2

1. Problem

We are tasked with finding an efficient CPP (Chinese Postman Problem) closed walk route within a DLH Dump Truck's service area. The route must traverse every single street (edge) in a directed road network (Surabaya) at least once with the minimum total travel distance.

2. Data

The map used for the road network is sourced from OpenStreetMap, filtered by highway classifications (trunk, primary, secondary, tertiary). The map is modeled as a

Directed MultiGraph (*DiGraph*) where edges store physical road lengths and names.

3. **Algorithms**
   - Eulerian Circuit & Degree Imbalance Check

     Before finding a route, the algorithm checks whether the directed graph is already Eulerian (balanced in-degrees and out-degrees for all nodes) and identifies unbalanced nodes.

     The algorithm iterates over every vertex in the road network graph `G`, computing the difference between its out-degree and in-degree. Nodes with excess outgoing edges are categorized as `supply` nodes, while nodes with excess incoming edges are categorized as `demand` nodes. If both dictionaries are empty, the graph is already Eulerian.

     From chinese_postman.py

```
out_deg = dict(G.out_degree())
in_deg = dict(G.in_degree())

supply = {}
demand = {}

for node in G.nodes():
    diff = out_deg.get(node, 0) - in_deg.get(node, 0)
    if diff > 0:
        supply[node] = diff # out > in (supply)
    elif diff < 0:
        demand[node] = -diff # in > out (demand)
```

   - Min-Cost Flow Augmentation

Using NetworkX's `min_cost_flow` solver, we can find the shortest paths connecting unbalanced nodes, augmenting the graph into a Eulerian graph. A transportation network flow model is constructed using a `super_source` and a `super_sink`, combined with Dijkstra's all-pairs shortest paths.

From chinese_postman.py

```
G_aug = G.copy()

if supply and demand:
    flow_net = nx.DiGraph()
    source, sink = "super_source", "super_sink"
    flow_net.add_node(source)
    flow_net.add_node(sink)

    total_supply = sum(supply.values())
    for s, qty in supply.items():
        flow_net.add_edge(source, s, capacity=qty, weight=0)
    for d, qty in demand.items():
        flow_net.add_edge(d, sink, capacity=qty, weight=0)

    lengths = dict(nx.all_pairs_dijkstra_path_length(G, weight='length'))
    paths = dict(nx.all_pairs_dijkstra_path(G, weight='length'))

    for s in supply.keys():
        for d in demand.keys():
            if d in lengths.get(s, {}):
                flow_net.add_edge(s, d, capacity=float('inf'), weight=lengths[s][d])

    try:
        flow_dict = nx.min_cost_flow(flow_net, demand={sink: total_supply})
        for s in supply.keys():
            if s in flow_dict:
                for d, flow_val in flow_dict[s].items():
                    if d in demand.keys() and flow_val > 0:
                        path = paths[s][d]
                        for i in range(len(path) - 1):
                            u, v = path[i], path[i+1]
                            edge_data = G.get_edge_data(u, v)
                            if edge_data:
                                key = list(edge_data.keys())[0]
                                G_aug.add_edge(u, v, **edge_data[key])
    except nx.NetworkXException:
```

- Hierholzer's Algorithm
  Once the graph has been augmented into an Eulerian directed graph, Hierholzer's algorithm extracts the final closed traversal circuit starting from the base of the DLH Dump Truck. NetworkX's built-in `eulerian_circuit` routine executes Hierholzer's

algorithm in linear time relative to the number of edges, outputting the complete sequence of street segments the garbage truck must traverse.

```
# Find Eulerian circuit
try:
    route_edges = list(nx.eulerian_circuit(G_aug, source=start_node))
except nx.NetworkXError:
    route_edges = [(u, v) for u, v in G_aug.edges()]

return G_aug, route_edges
```

4. Complexity

The Degree Imbalance Checker has a time complexity of (O(V + E)), looping through all vertices V and edges E to compute the out-degree and in-degree. Dijkstra's All-Pairs Shortest Paths (APSP) has a time complexity of (O($V^2 logV + VE$)), it is run across the graph in order to find the shortest path between every pair of unbalanced supply and demand nodes. Min-Cost Flow Augmentation's time complexity is (O(V^3)), it has a number of nodes proportional to the number of supply and demand nodes (bounded by V) and edges bounded by (V^2)(every supply connected to every demand node). Solving min-cost flow on the network adds a polynomial term, which is bounded or dominated by Dijkstra's APSP phase on typical road networks. Hierholzer's algorithm has a time complexity of (O(E)), it traverses every edge of the augmented graph exactly once to build the final circuit runs in linear time relative to the number of edges (E).

Because Dijkstra's APSP's time complexity of (O($V^2 logV + VE$)) dominates the execution time over the simple degree checks and the linear graph traversals, the overall time complexity is expressed as (O($V^2 logV + VE$)).

E. Task 3

1. Problem

We are tasked with finding the sequence of moves of a knight on a chessboard where the knight visits every square exactly once.

2. Data

The chessboard is modeled as a 2D grid coordinate system (column x row) where each square acts as a graph node connected to up to 8 valid knight move destinations.

From knight_tour.py

```
_MOVES = (
    (1, 2),
    (2, 1),
    (2, -1),
    (1, -2),
    (-1, -2),
    (-2, -1),
    (-2, 1),
    (-1, 2),
)
```

3. Algorithms

- Move Ordering using Warnsdorff's Heuristic

Warnsdorff's rule dictates that the knight should always move to the adjacent square that has the fewest onward valid moves, greedily avoiding premature dead-ends.

From knight_tour.py

```
def candidates(col: int, row: int) -> list[tuple[int, int]]:
    available = []
    for col_delta, row_delta in _MOVES:
        next_col = col + col_delta
        next_row = row + row_delta
        if (
            0 <= next_col < cols
            and 0 <= next_row < rows
            and not visited[next_col][next_row]
        ):
            # Warnsdorff counts each candidate's available onward moves.
            onward = sum(
                0 <= next_col + move_col < cols
                and 0 <= next_row + move_row < rows
                and not visited[next_col + move_col][next_row + move_row]
                for move_col, move_row in _MOVES
            )
            available.append((onward, next_col, next_row))
    available.sort()
    return [(next_col, next_row) for _, next_col, next_row in available]
```

- DFS Search & Backtracking Core
  The recursive search function explores candidate moves sequentially. If a dead-end is reached, it backtracks by unmarking the visited state and popping from the path stack.

From knight_tour.py

```
def search(col: int, row: int) -> bool:
    visited[col][row] = True
    path.append((col, row))
    # The base case means every board square has been visited.
    if len(path) == cols * rows:
        return True

    for next_col, next_row in candidates(col, row):
        # Each recursive call explores one candidate branch.
        if search(next_col, next_row):
            return True

    path.pop()
    # Unmarking lets another branch reuse this square during backtracking.
    visited[col][row] = False
    return False
```

4. Complexity

- Time Complexity:
  Without heuristic guidance, exploring all potential paths yields a worst-case exponential time of $O(8^N)$, where N = cols x rows is the number of squares. But with Warnsdorff's, the heuristic prunes the search tree so effectively that the algorithm runs in near-linear O(N) time, solving standard chessboards in milliseconds without hitting deep recursive dead-ends.
- Space Complexity:
  O(N), the memory footprint is proportional to the total number of squares N due to the 2D visited matrix lookup table, the coordinate recursion call stack, and the final path coordinate list.